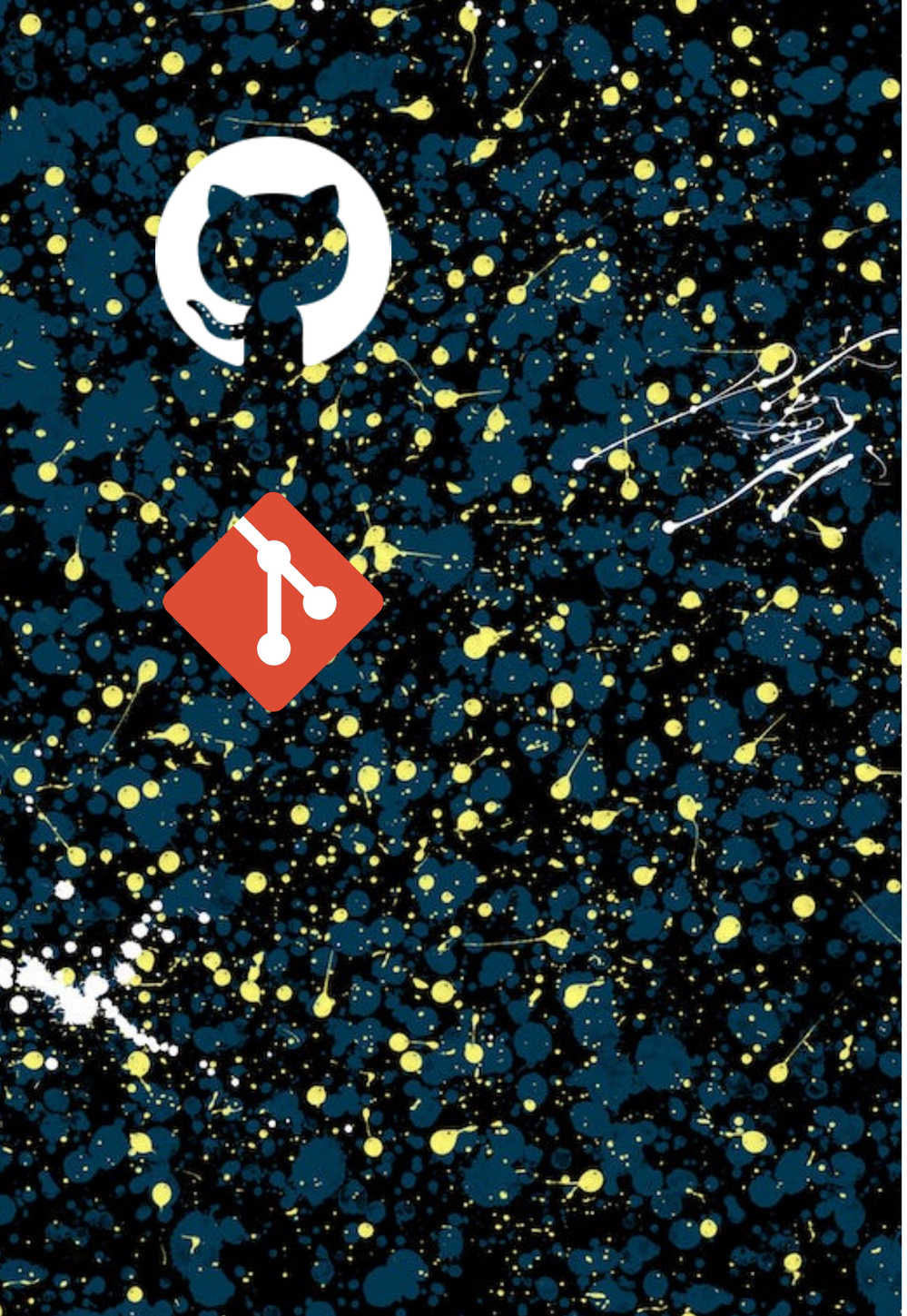
ダウンロード・取消・コンフリクト



ダウンロード関連

リモートからのダウンロード

git clone 丸ごとダウンロード

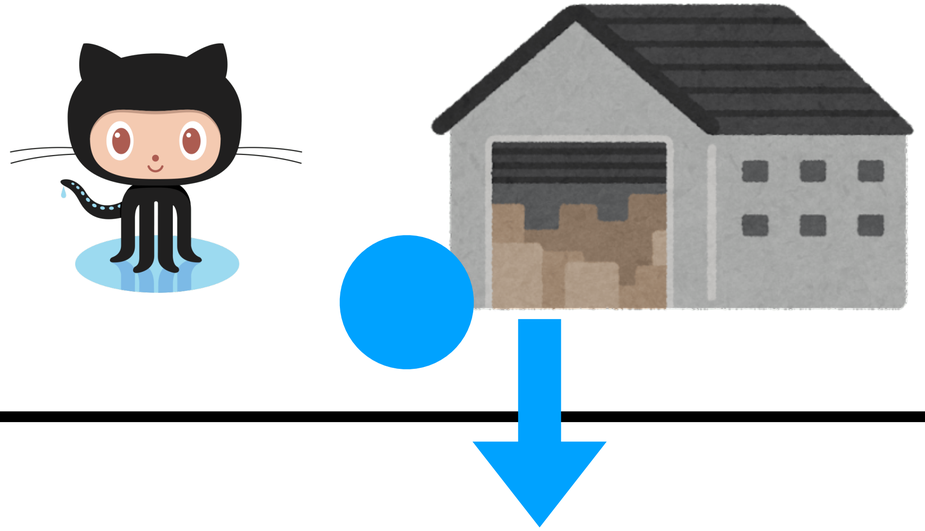
git pull 差分ダウンロードしてマージ

git fetch 差分確認

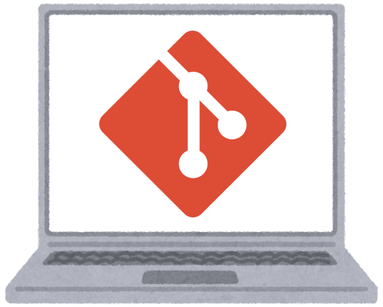
git clone 丸ごとダウンロード

git clone <gitHubのurl>

Remote



Local

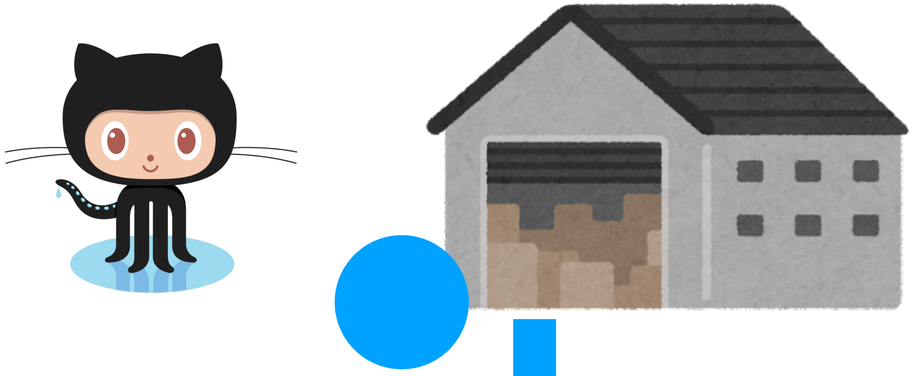


フォルダだけ
つくっておく

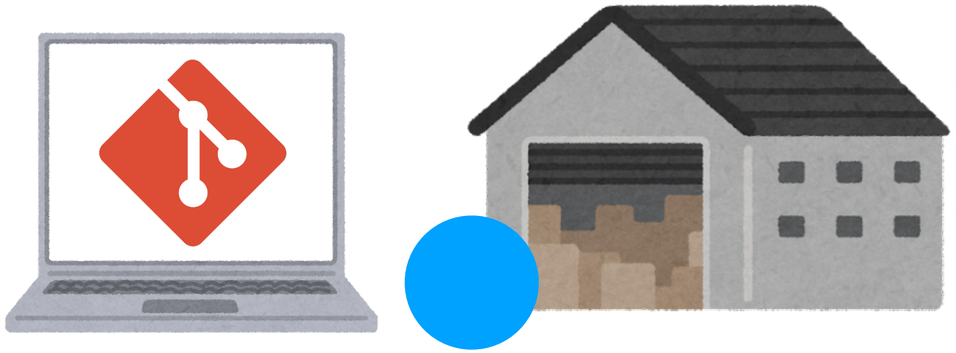
git clone 丸ごとダウンロード

git clone <gitHubのurl>

Remote



Local

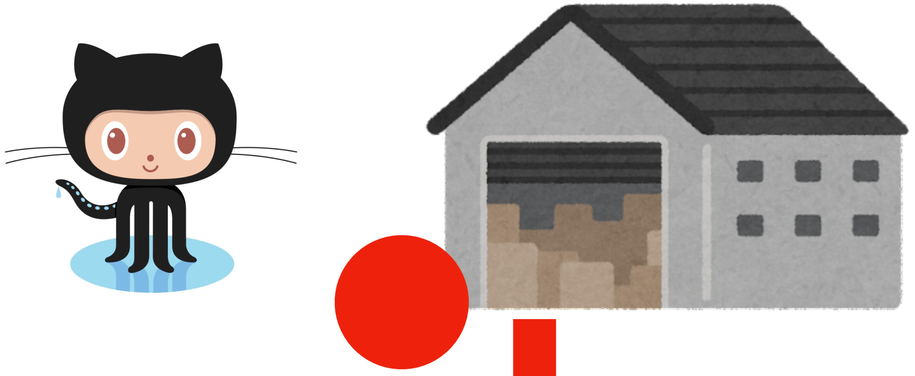


リポジトリを生成
デフォルトブランチを
ダウンロード

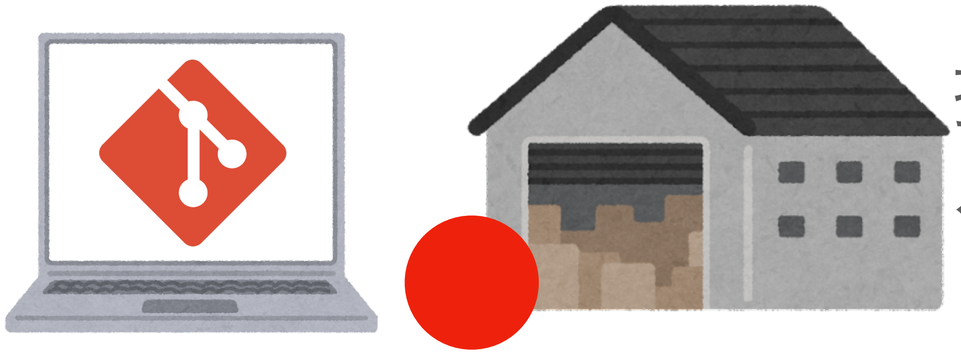
git clone ブランチ指定

git clone -b <ブランチ名> <gitHubのurl>

Remote



Local

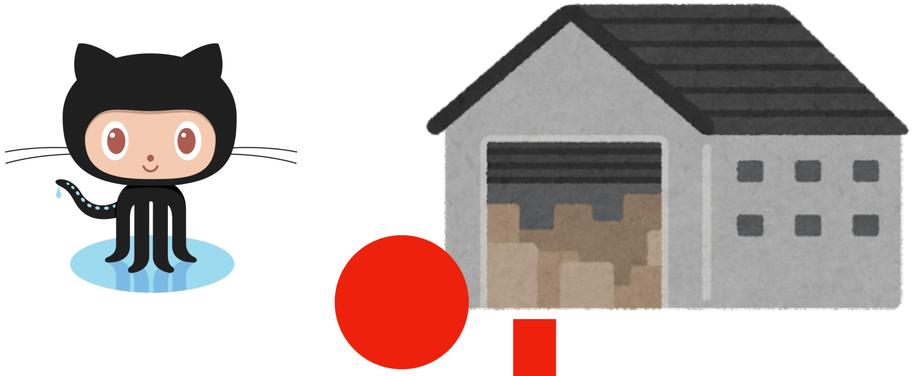


リポジトリを生成
指定ブランチを
ダウンロード

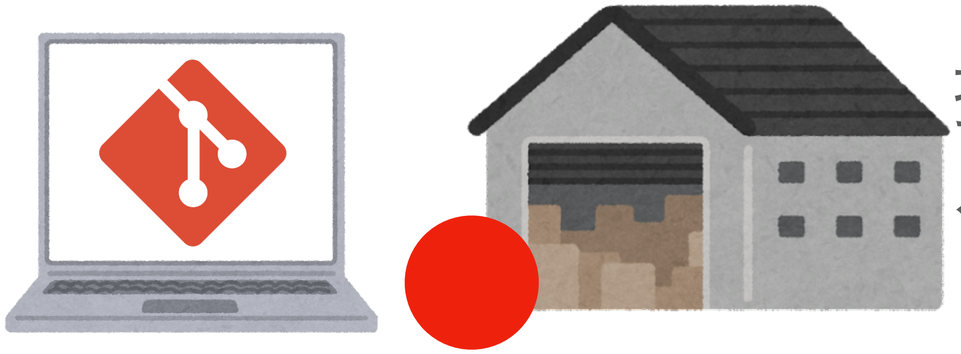
git clone ブランチ指定フォルダ指定

git clone -b <ブランチ名> <gitHubのurl> フォルダ名

Remote



Local

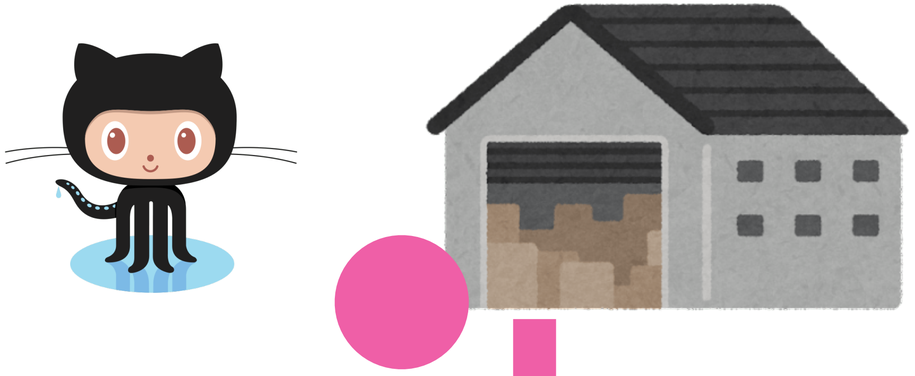


リポジトリを生成
指定ブランチを
ダウンロード

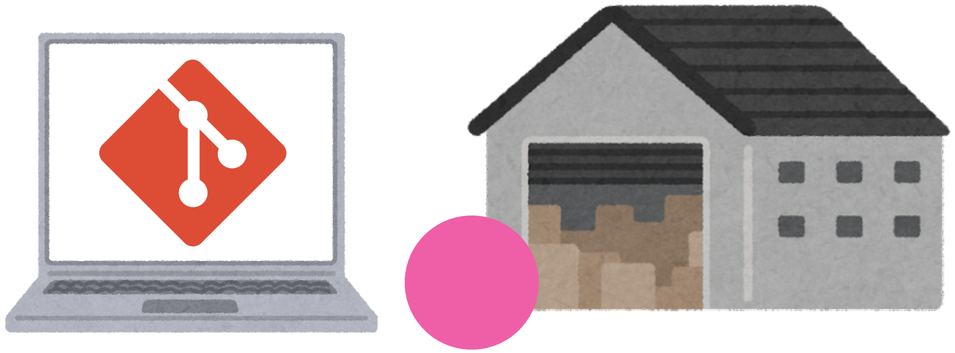
git pull (fetch + merge)

git pull origin main

Remote



Local



差分ダウンロード

+

マージ

動画の補足

mainブランチは
いくつかコンフリクト(衝突)が
あったため、

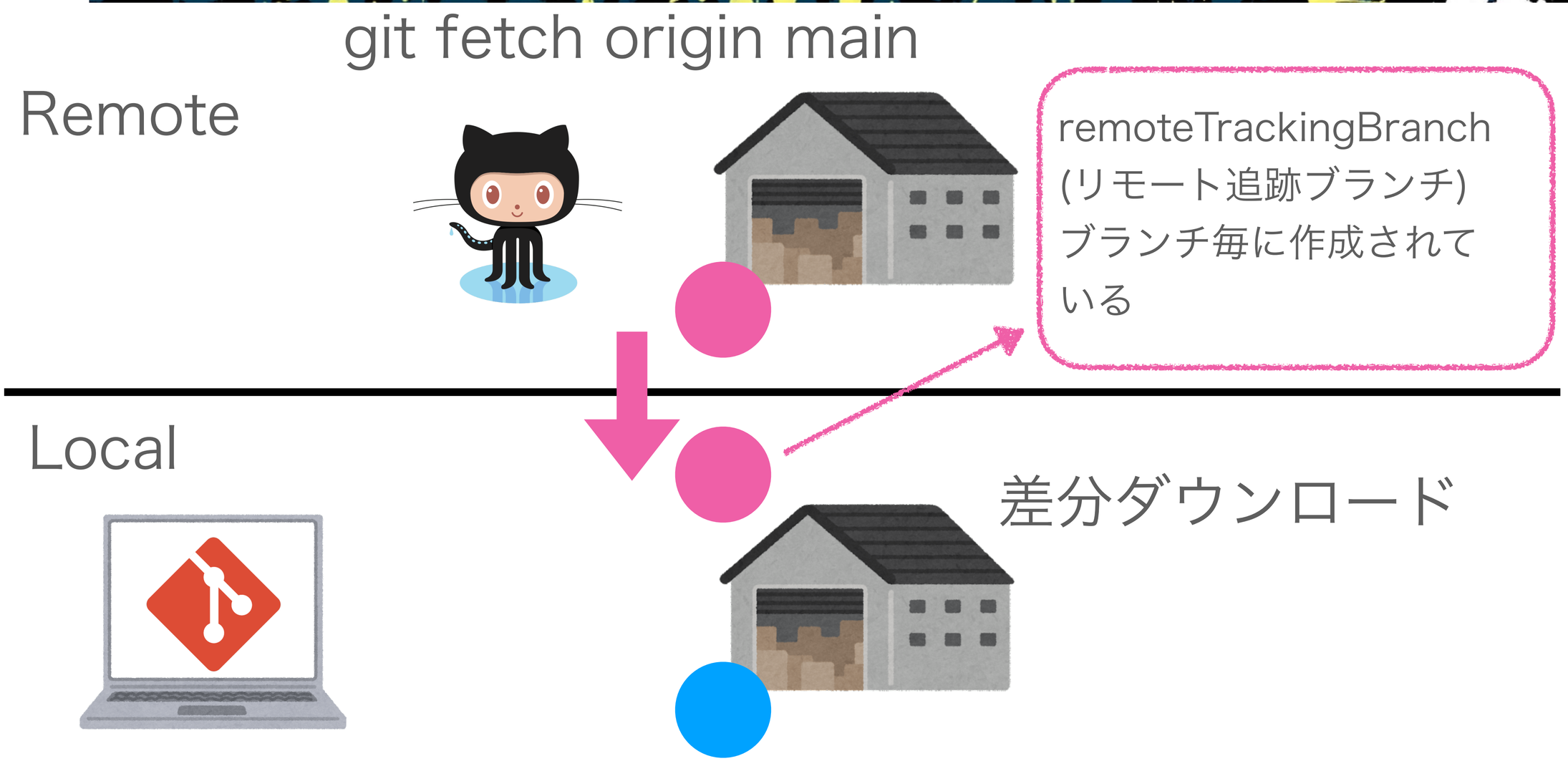
わかりやすさ重視という事で
practiceブランチに変更して
収録しております。

git pullのwarning

Git 2.27.0で導入されたメッセージ
rebase false //デフォルトの挙動
rebase true //--rebase実行(コミット履歴一直線)
ff only //fast-forward

```
From https://github.com/aokitashipro/git-sample-udemy
   cd70922..9adaa08  main       -> origin/main
hint: You have divergent branches and need to specify how to reconcile them.
hint: You can do so by running one of the following commands sometime before
hint: your next pull:
hint:
hint:   git config pull.rebase false  # merge
hint:   git config pull.rebase true   # rebase
hint:   git config pull.ff only       # fast-forward only
hint:
hint: You can replace "git config" with "git config --global" to set a default
hint: preference for all repositories. You can also pass --rebase, --no-rebase,
hint: or --ff-only on the command line to override the configured default per
hint: invocation.
fatal: Need to specify how to reconcile divergent branches.
```

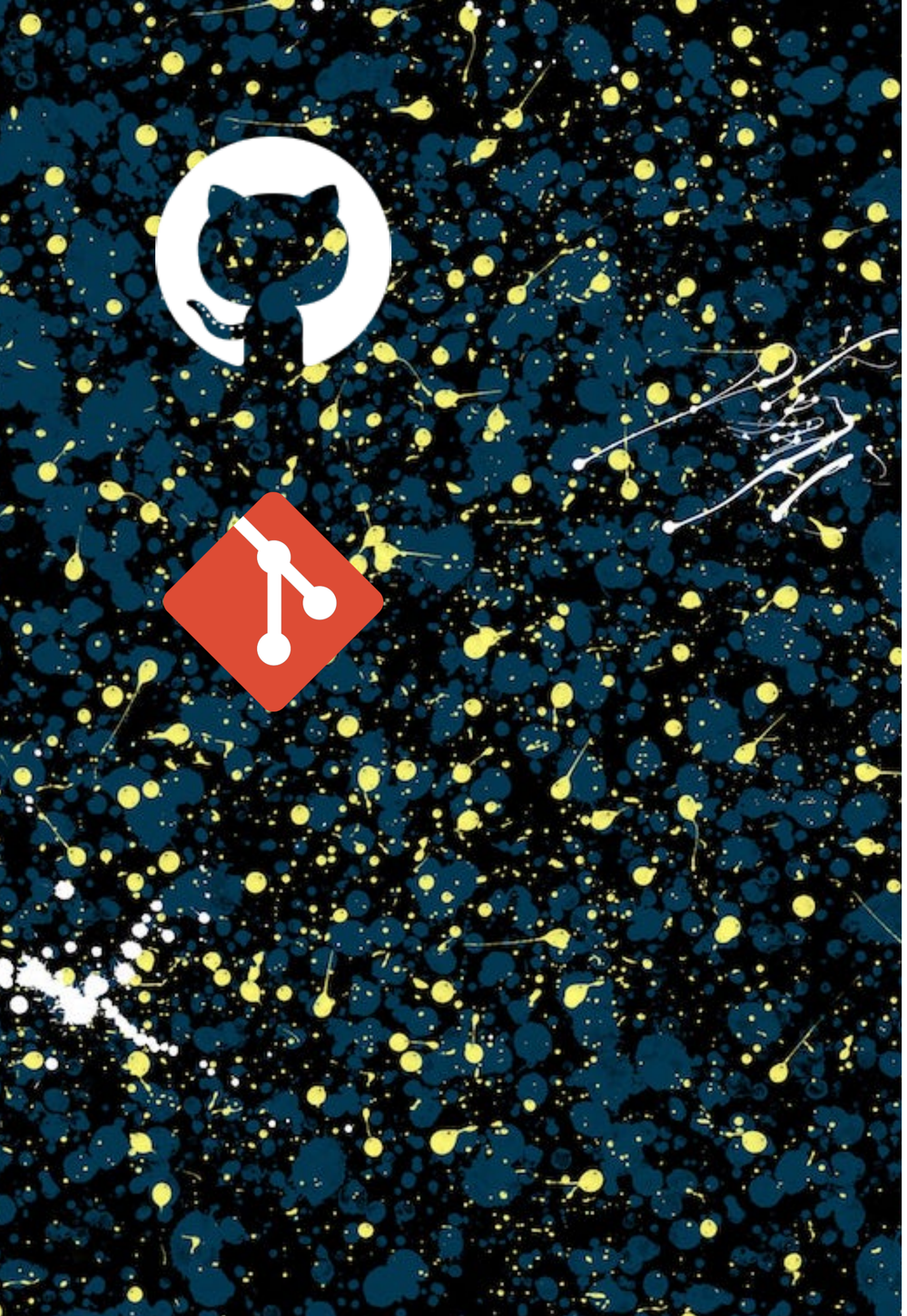
git fetch 差分ダウンロード



git fetchの確認方法

git fetch すると
リモート追跡リポジトリが更新される
git branch -a //ブランチ名確認
remotes/origin/main などが表示される

・差分の確認方法
 git diff origin/ブランチ名
 git log origin/ブランチ名 --oneline
・差分確認後のマージ
 git merge <ブランチ名>



取り消し系のコマンド

取り消しの種類

間違いだってあるじゃない、人間だもの

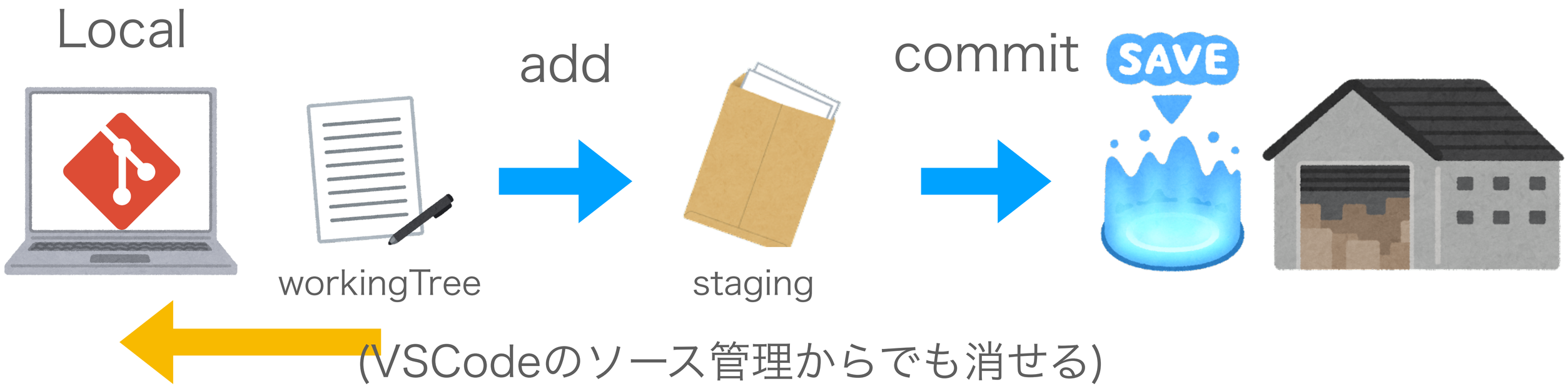
・取り消したい内容
addする前に書いたコード
git add
git commit
git push
pull request
merge pull request
git merge
commit コメントの修正

取り消し系のコマンド

git clean
git reset
git revert
git restore
(以前は git checkoutだったけれど
switchとrestoreに分離)

それぞれ複数のオプションがあります

addしてないコードを消したい



(VSCodeのソース管理からでも消せる)

・新規作成ファイルを消す

clean -f <ファイル名>

・ファイル変更を取り消す

restore <フォルダ/ファイル> // ファイル取消

restore . // add全て取消

addを取り消したい

(VSCodeのソース管理からでも消せる)

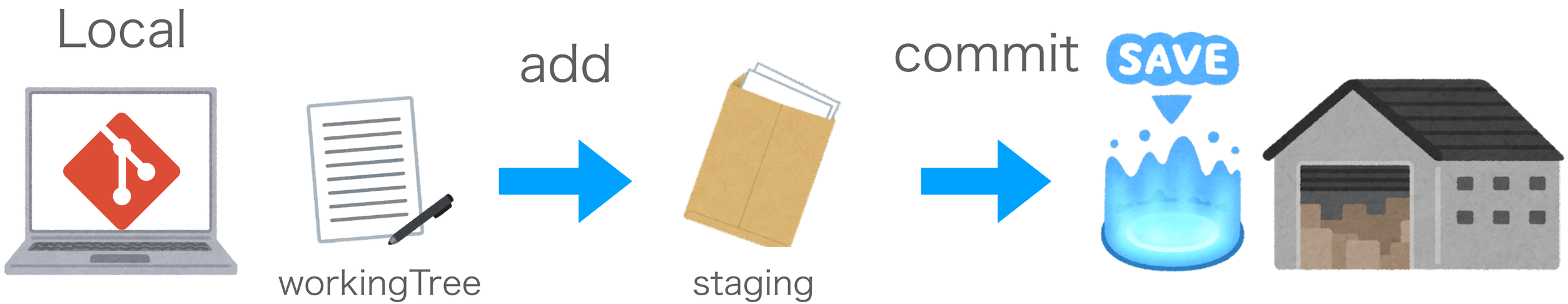
restore --staged <フォルダ/ファイル> // ファイル取消

restore --staged . // add全て取消

reset <ファイル> // ファイルの取消

reset // addを全て取消

commitを取り消したい



HEAD・・現在の場所
resetはcommitを
なかったことにする

reset --soft HEAD^

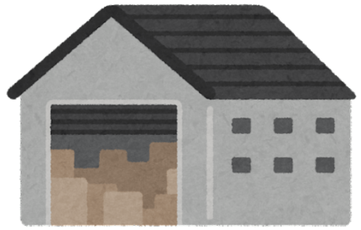
reset --mixed HEAD^

reset --hard HEAD^

コミットID指定時は
最新の1つ前の
IDを指定する

pushを取り消したい

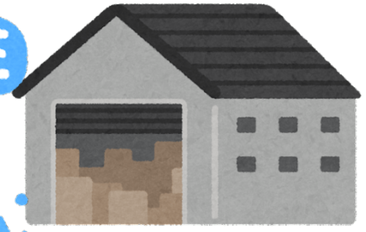
Remote



コミット履歴を残す場合

```
git revert <commitID>
git push // 取消コミットのプッシュ
```

Local

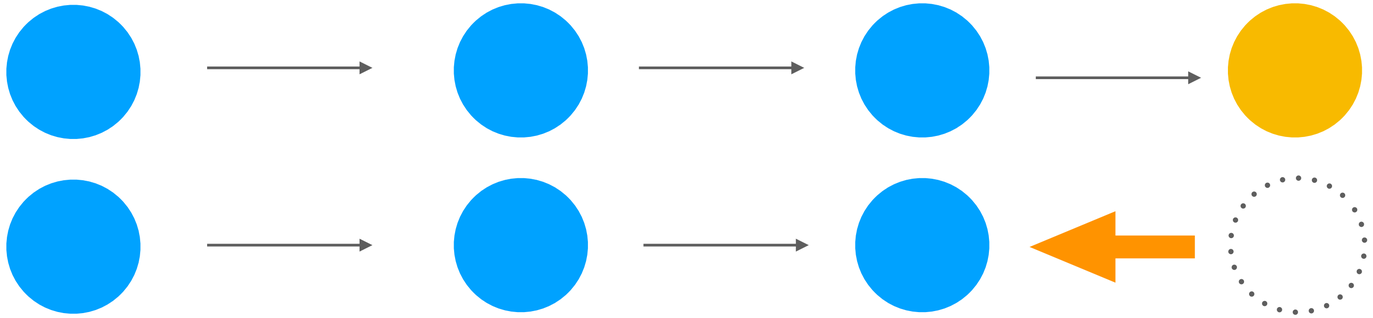


コミット履歴を残さない場合

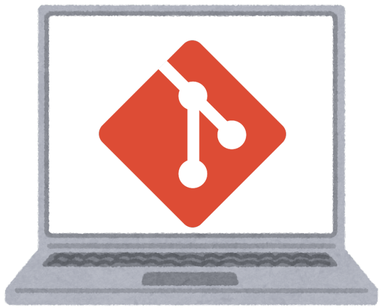
```
git reset --soft <commitID>
git push -f // 強制プッシュ
```

resetとrevert

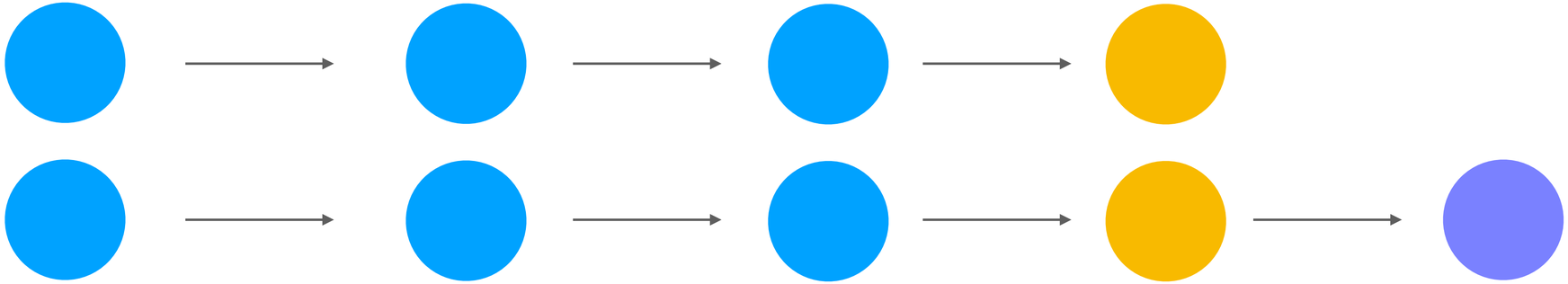
resetはコミット毎削除



Local

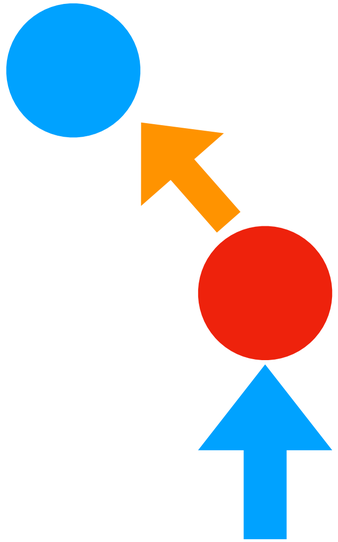


revertは打ち消しコミットをつくる



pull requestを取り消したい

Remote



Pull Request
(確認依頼)

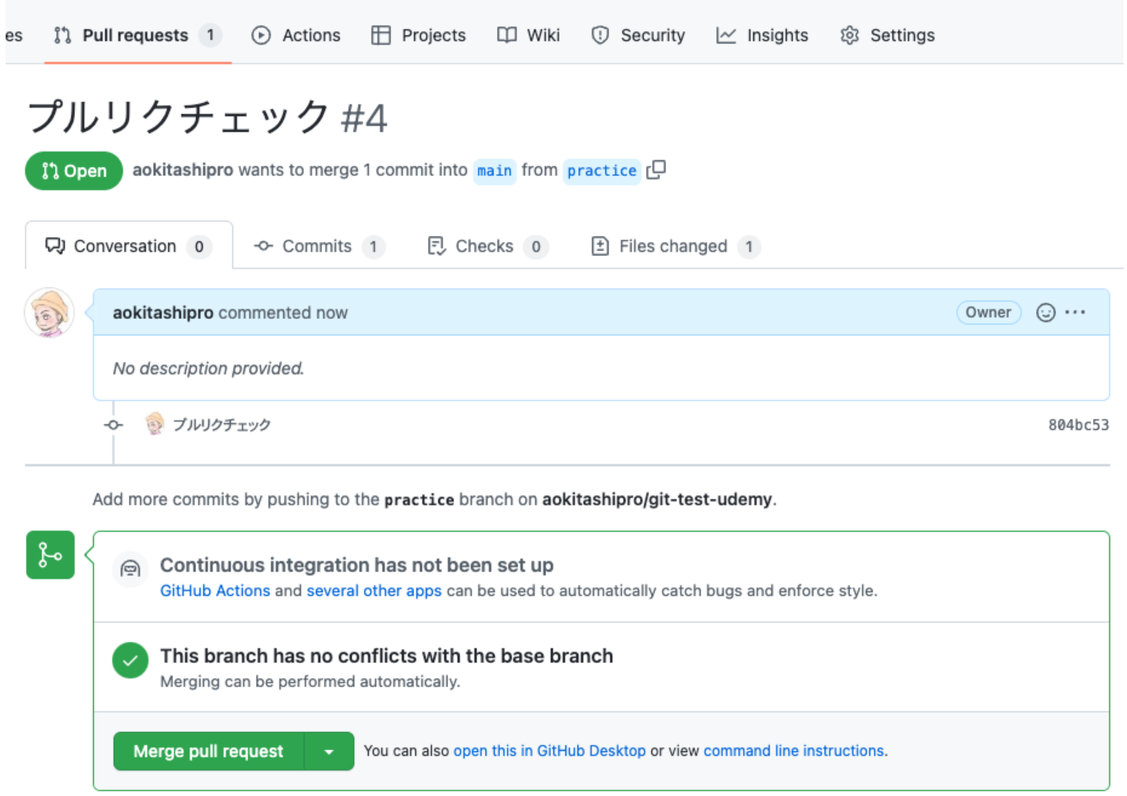
プルリク前後に取り消す (マージ前)

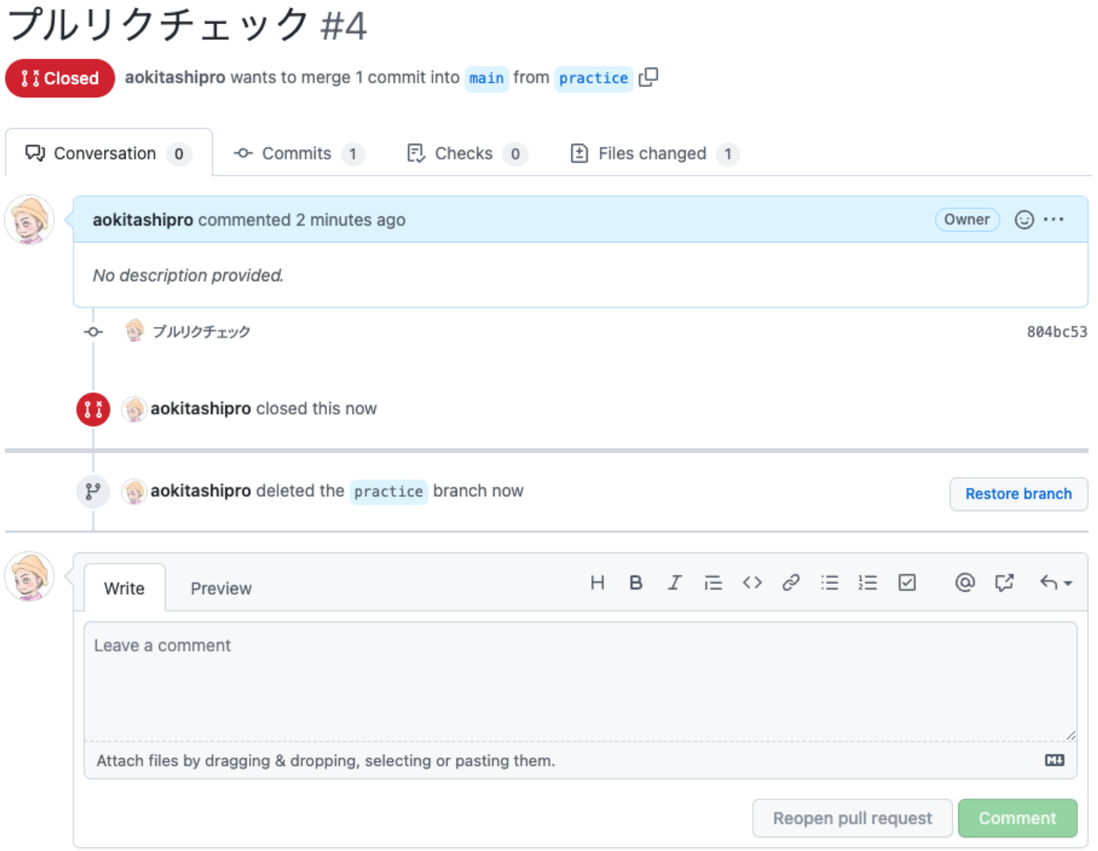
git push --delete origin <branch名>

pull requestを取り消したい

プルリク出す前

プルリク出した後 (取り消すとclosedになる)



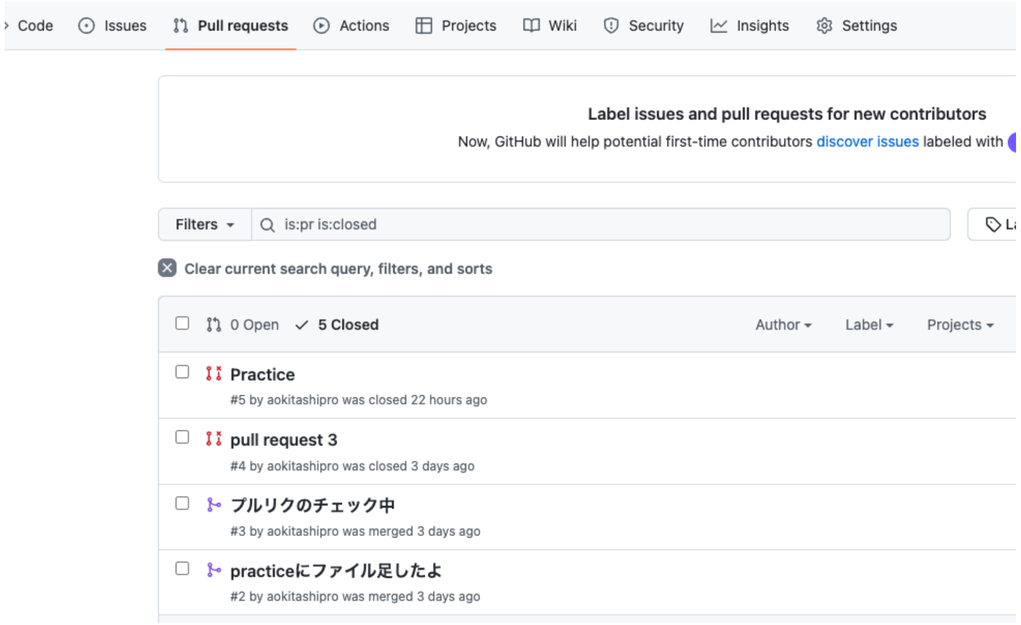


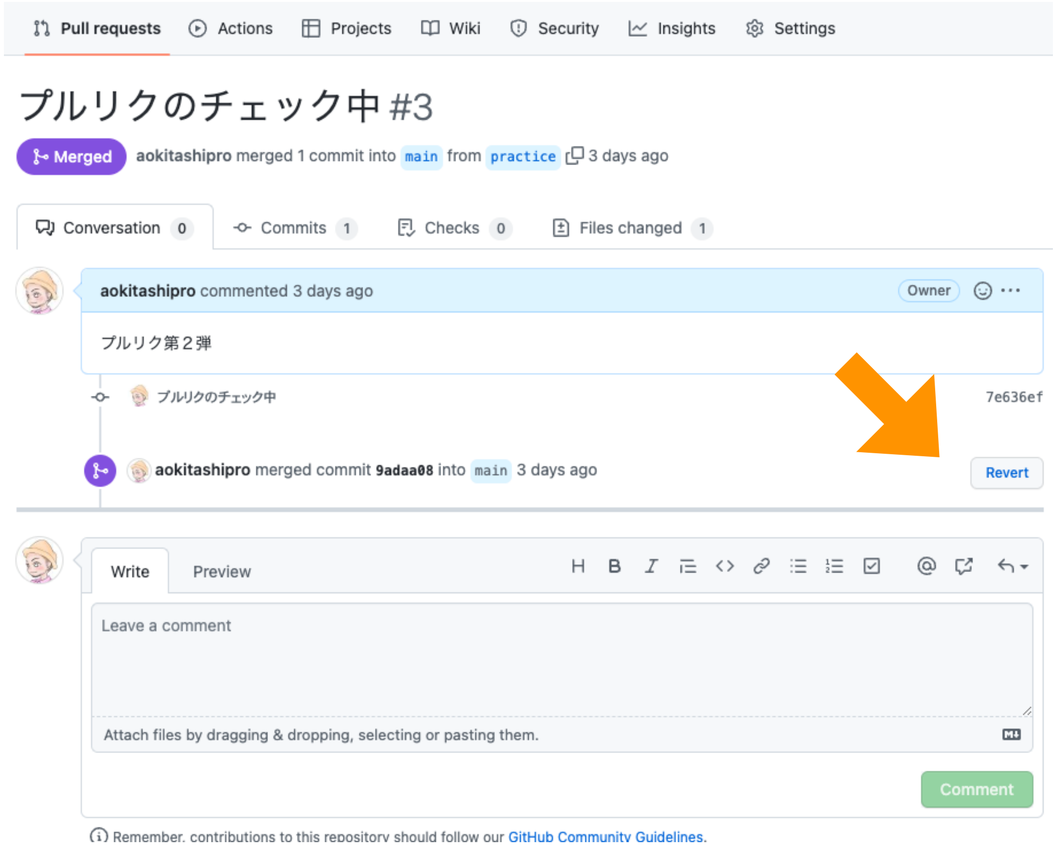
mergeしたプルリクを取消したい

Remote

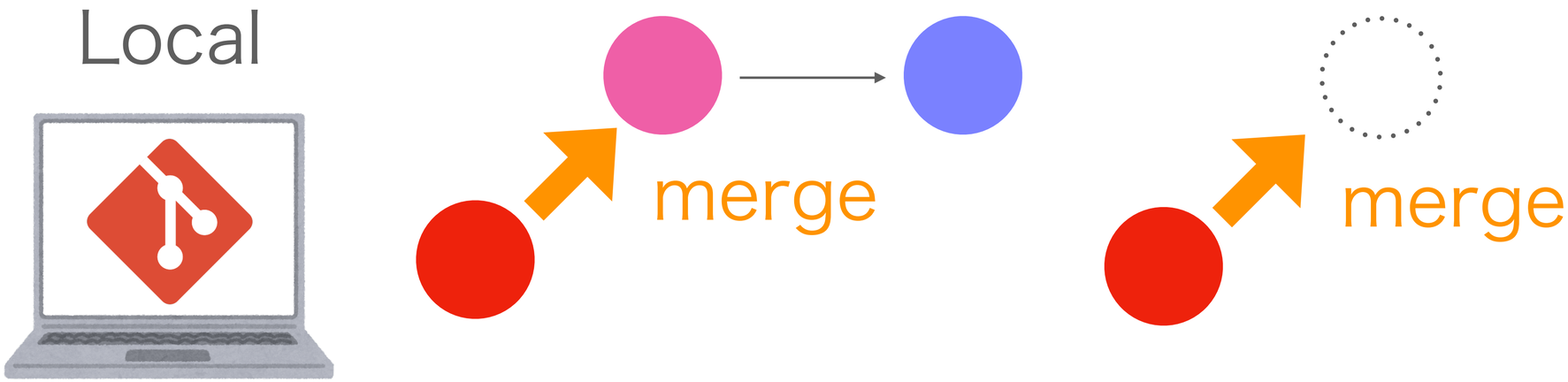
Pull Request(確認依頼)

& マージ





ローカルマージを取り消したい



マージコミット

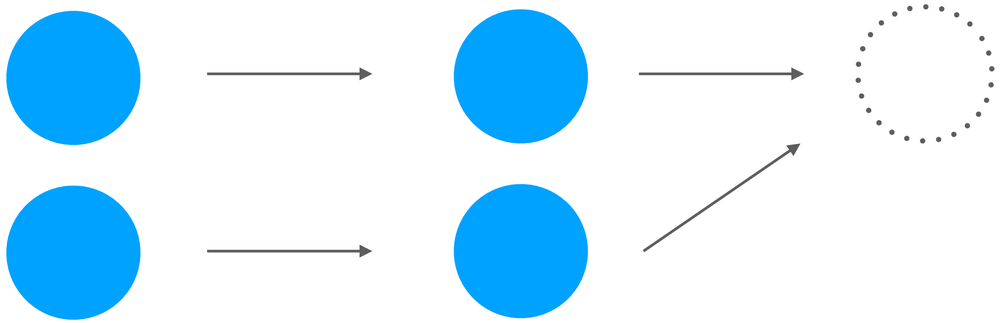
履歴を残す git revert -m 1 <コミットID>

(数字は git show や git log で確認)

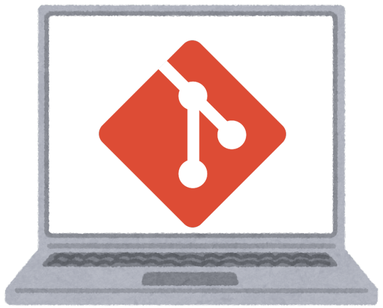
履歴を残さない git reset --hard ORIG_HEAD

resetとrevert

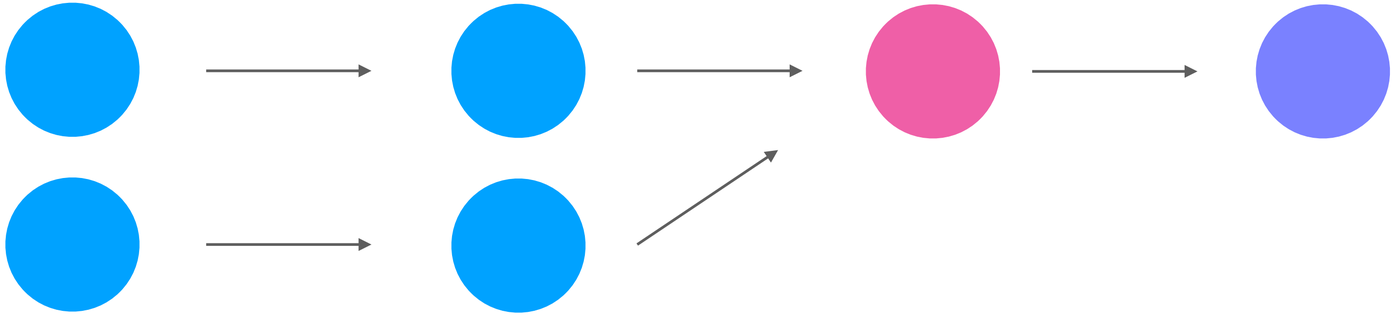
resetはコミット毎削除



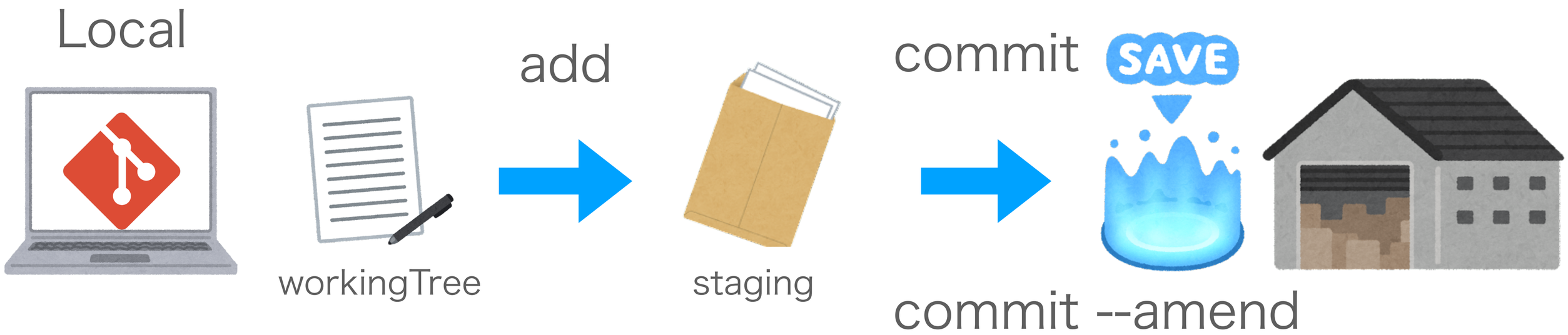
Local



revertは打ち消しコミットをつくる



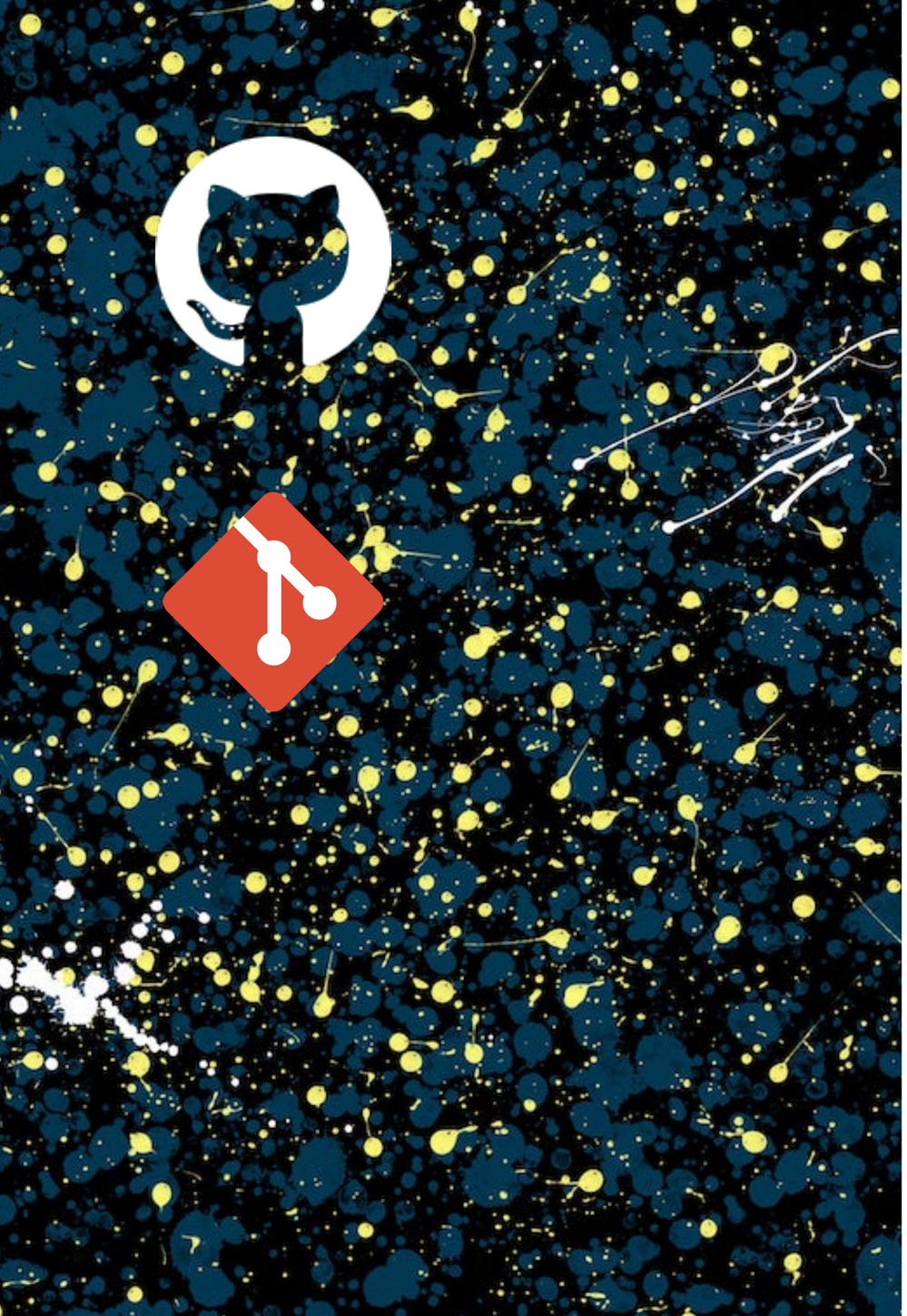
commitコメントを修正したい



最新コミットのコメントを修正する

commit --amend -m '修正後のコメント'

(push したものを変更するとコンフリクト発生するので要注意)

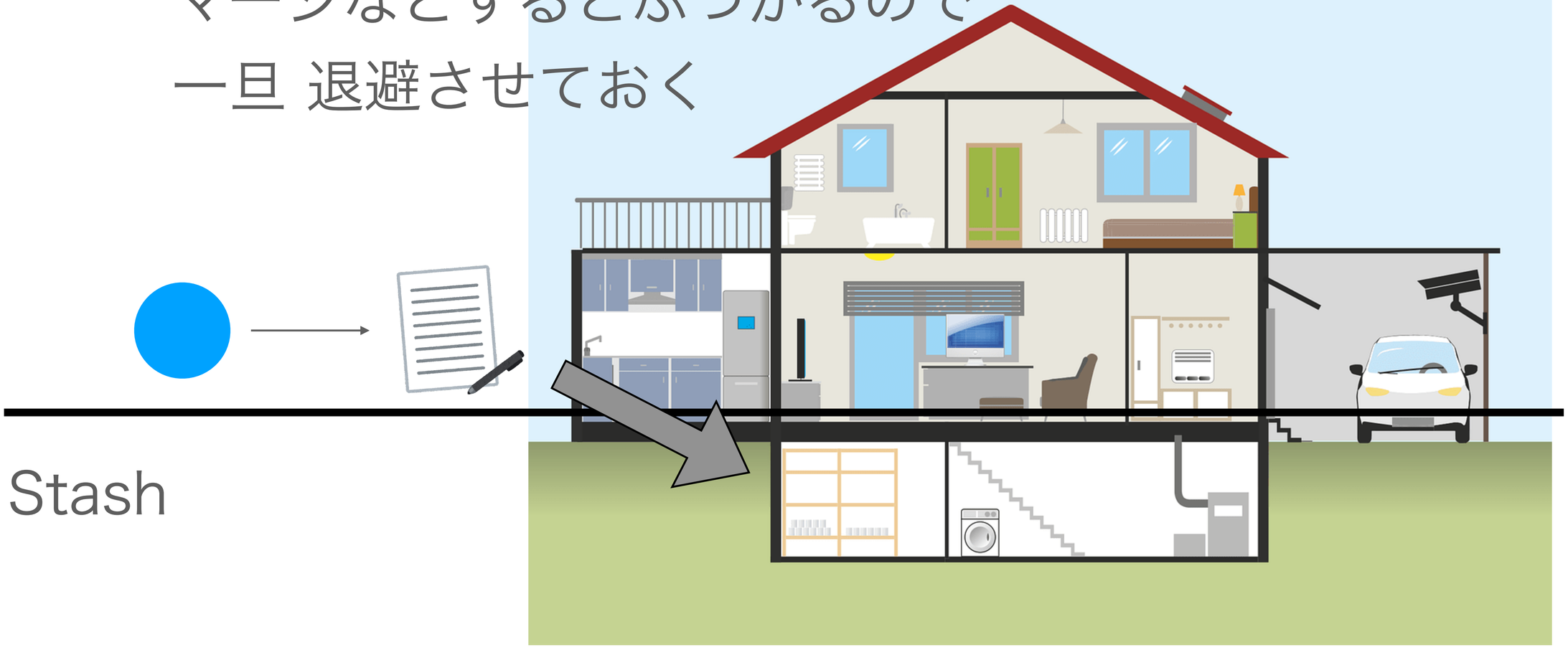


スタッシュ
(隠し場所)

スタッシュ (隠し場所)

Local

コミット後コードを書いていて
マージなどするとぶつかるので
一旦 退避させておく



Stash

git stashのオプション

git stash
git stash save <コメント>
git stash save -u <コメント> //未追跡も退避

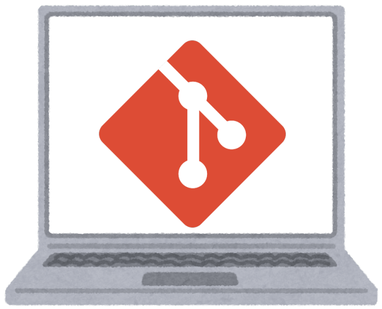
git stash list // stashの表示
git show stash@{1} 1つ前の内容

stash apply, drop, pop

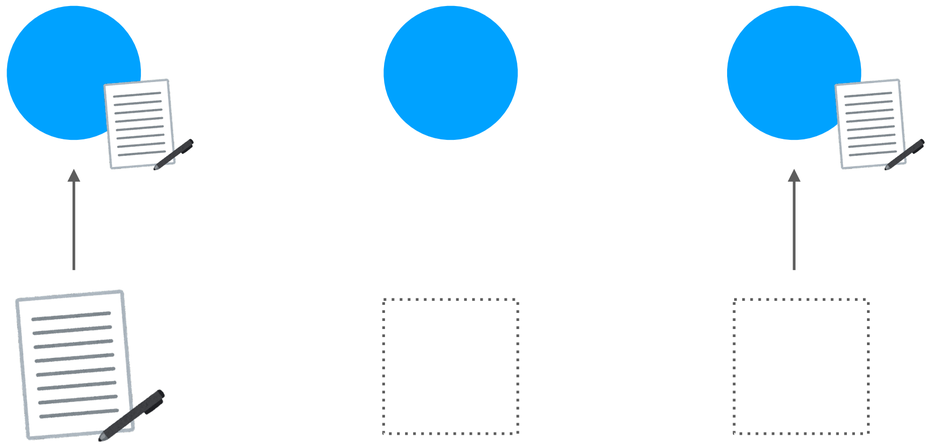
stash apply　　stash pop

stash drop

Local

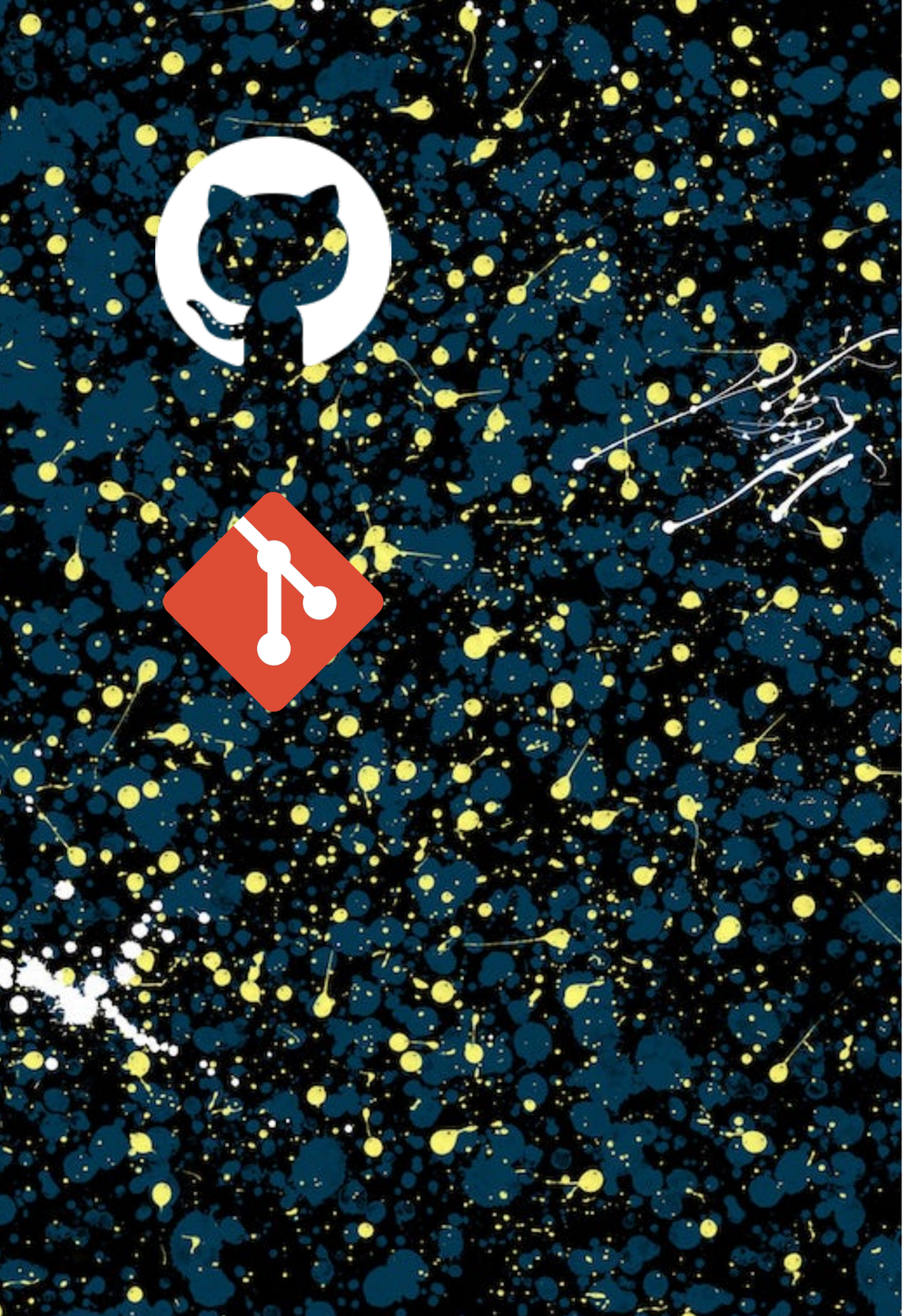


Stash



stash xxx stash@{1}　何番目かを指定できる

番号指定がなければ最新のstashが対象になる



コンフリクト(衝突)

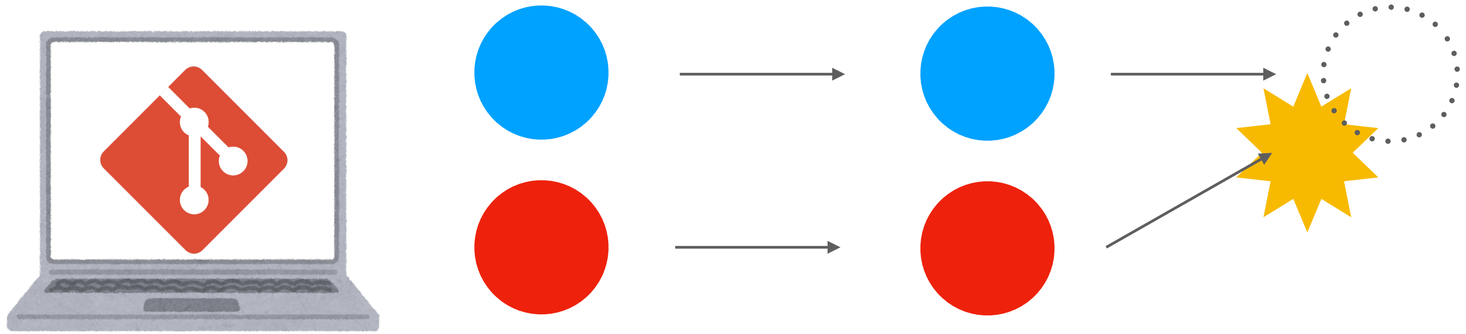
コンフリクト(衝突)

同じファイル、同じ行に違う内容が書かれていてマージする際に発生

ローカルからローカル
ローカルからリモート (pushできない)
リモートからローカル (pullできない commit後)
リモートからローカル (pullできない add後)
スタッシュからローカル
リモートからリモート (プルリク)

ローカルからローカル

ローカルの別ブランチをマージしようとして衝突



git statusで状況、必要コマンドなどを確認できる

ローカルからローカル

```
Accept Current Change | Accept Incoming Change | Accept Both Changes | C…
1 <<<<<<<·HEAD (Current Change)
2 mainブランチで書いてます
3 =======
4 practiceブランチで書いています
5 >>>>>>>·practice (Incoming Change)
6
```

同じファイル・同じ行に違う内容で表示
Accept Current Changeは
現在の内容を優先する

>>>>>などを手動で消してもok

修正後は
git add .
git commit

```
問題　出力　デバッグ コンソール　ターミナル　zsh
  switch-to-practice
a-aoki@aaokiMBP git-sample-udemy % git merge practice
CONFLICT (modify/delete): cancel/merge.html deleted in HEAD
 and modified in practice.  Version practice of cancel/merg
e.html left in tree.
Automatic merge failed; fix conflicts and then commit the r
esult.
a-aoki@aaokiMBP git-sample-udemy %
aph　Go Live
```

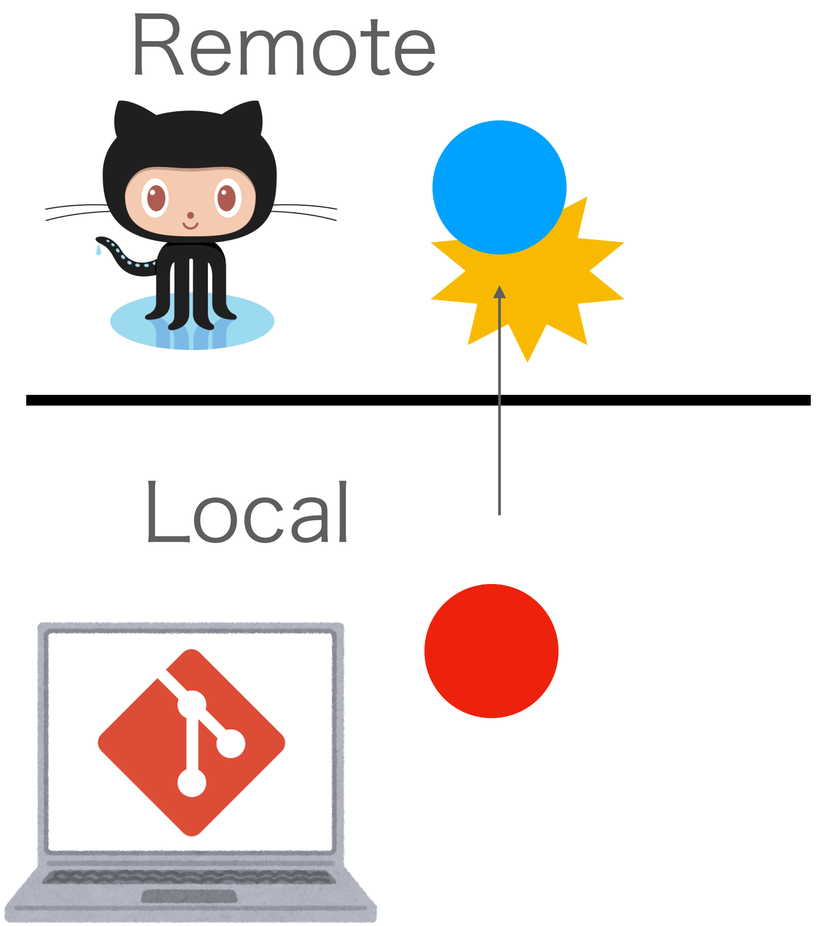
以前 local mergeをrevertで
打ち消していたので
差分が存在
git status で確認して
git add. で追加でok

ローカルとリモート 一旦あわせる

わかりやすくするために
一旦 ローカルとリモートの状態を
合わせることにします。

現在ローカルの方が進んでいるため,
各ブランチで git push -u origin <ブランチ名>
として強制pushしファイル内容を合わせてから
先へ進むことにしています。

ローカルからリモート (push)



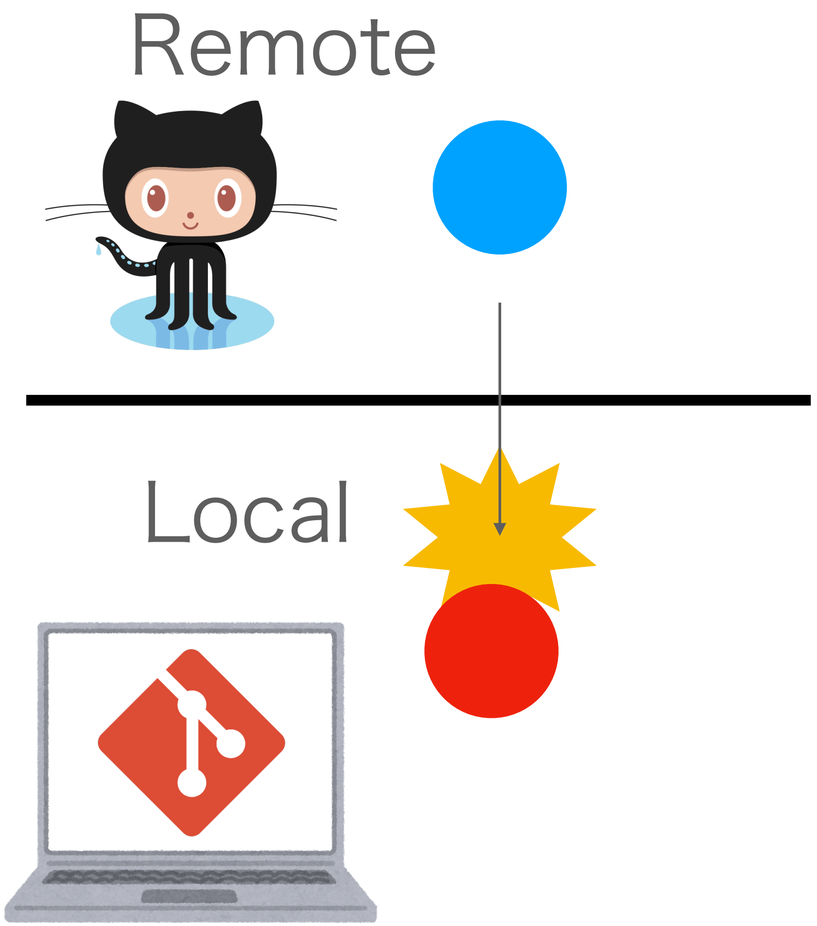
ローカルとリモートに差分がある状態で
push しようとすると 拒否 される

```
a-aoki@aaokiMBP git-sample-udemy % git push
To https://github.com/aokitashipro/git-sample-udemy.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/aokitashipro/git-
sample-udemy.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pus
hing
hint: to the same ref. You may want to first integrate the remote change
s
hint: (e.g., 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for detail
s.
a-aoki@aaokiMBP git-sample-udemy %
```

git pull実施するとマージ確認画面になり

修正後改めて git push でアップロード可能

リモートからローカル (pull)

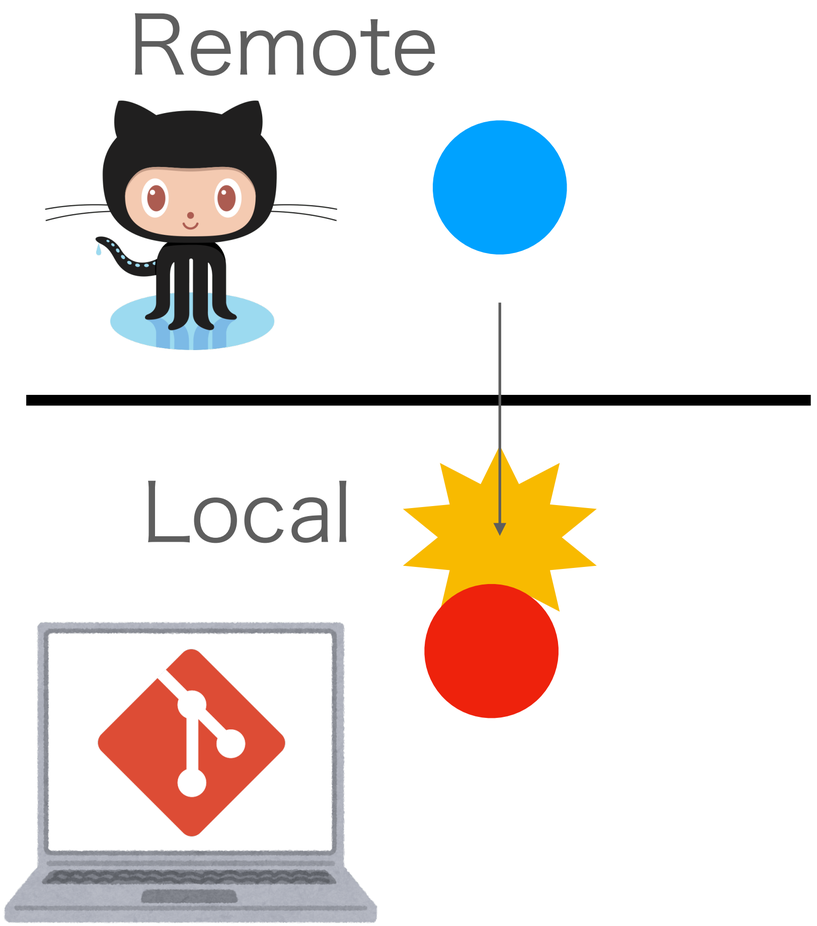


ローカルでcommit済みの場合

マージ確認画面になり、
マージ後
改めてpull でok

リモートからローカル (pull)

ローカルでadd済み commit前の場合




```
a-aoki@aaokiMBP git-sample-udemy % git pull
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (4/4), 1.01 KiB | 516.00 KiB/s, done.
From https://github.com/aokitashipro/git-sample-udemy
   ddcbfd9..ab251c1  practice   -> origin/practice
Updating ddcbfd9..ab251c1
error: Your local changes to the following files would be overwritten by m
erge:
        conflict/pull.html
Please commit your changes or stash them before you merge.
Aborting
a-aoki@aaokiMBP git-sample-udemy %
```

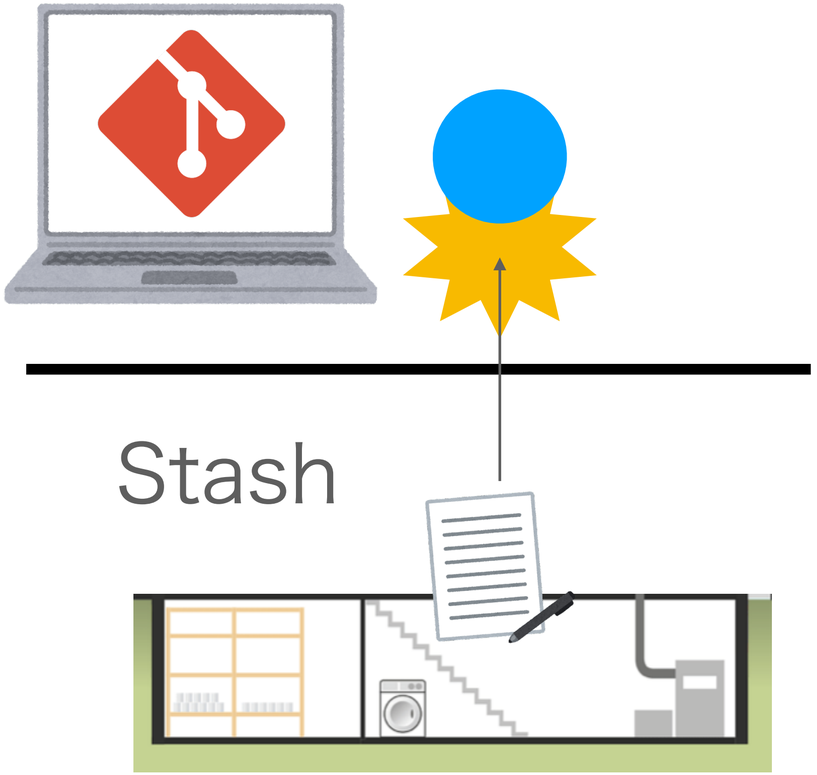
commitしてない分が消えるので

commitするかstashで

一旦退避するか

スタッシュからローカル

Local



スタッシュから戻す際に衝突

```
a-aoki@aaokiMBP git-sample-udemy % git stash pop
Auto-merging conflict/pull.html
CONFLICT (content): Merge conflict in conflict/pull.html
On branch practice
Your branch is up to date with 'origin/practice'.

Unmerged paths:
  (use "git restore --staged <file>..." to unstage)
  (use "git add <file>..." to mark resolution)
        both modified:   conflict/pull.html

no changes added to commit (use "git add" and/or "git commit -a")
The stash entry is kept in case you need it again.
a-aoki@aaokiMBP git-sample-udemy %
```

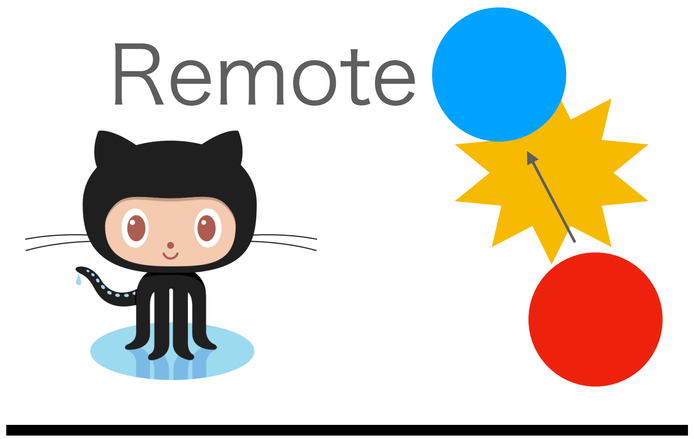
aph Go Live

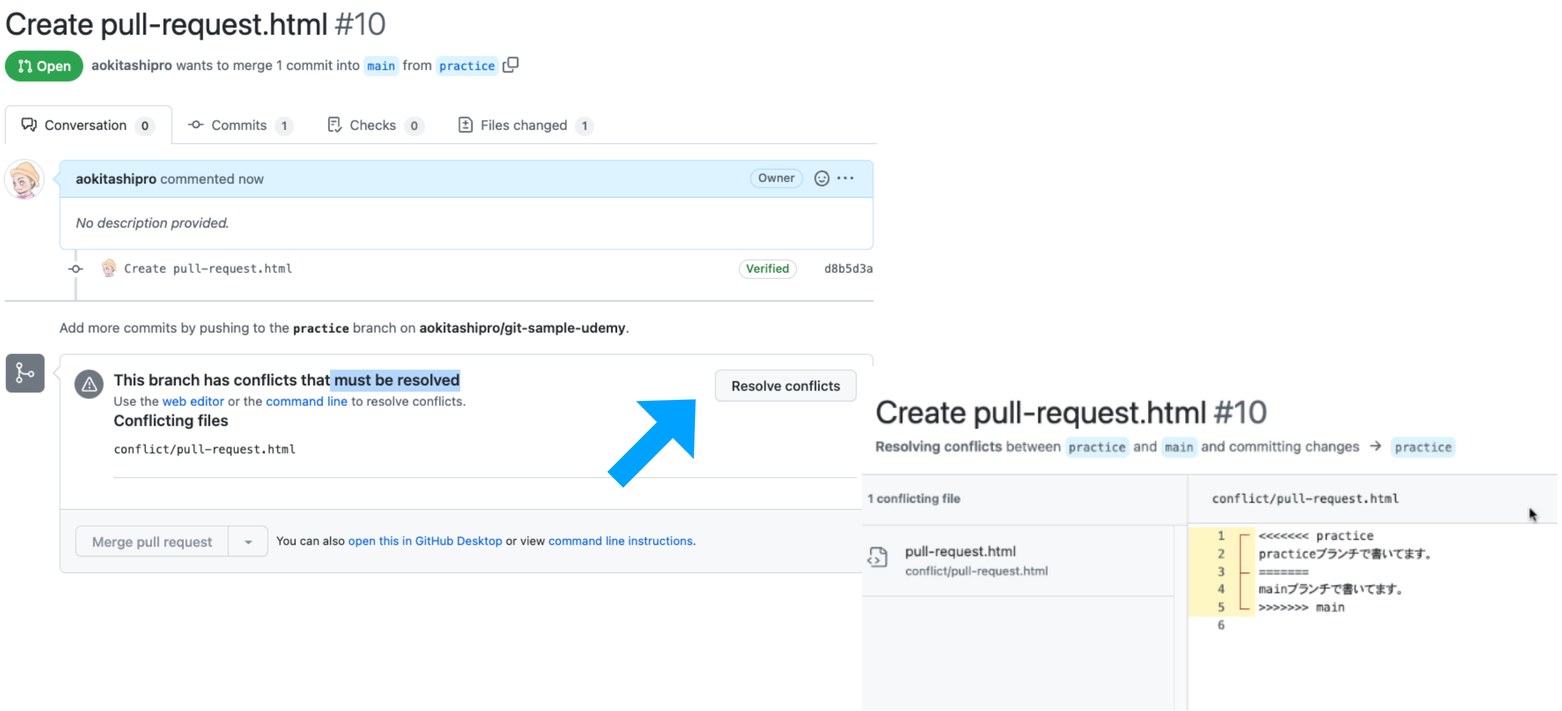
マージ画面で対応後

git add . git commit など実施

リモートからリモート(プルリク)

プルリク作成後 コンフリクト発生





Resolve conflictsをクリックし

各ファイルを修正後マージ可能